

An Introduction to Numerical Analysis

J. L. Clements

Chapter 1. Real numbers and floating point

Numerical analysis lives in the gap between the real numbers, with which mathematics is conducted, and the finite subset of those numbers that a computer can actually represent. This chapter develops the IEEE 754 double-precision floating-point model¹, the basic error notions (absolute, relative, machine epsilon), and the conditions under which a numerical algorithm can be said to *converge*. The treatment is informal but standard; readers who prefer the rigorous account should consult Higham [1] or the original IEEE specification [2].

1985 original with decimal arithmetic and a small number of new operations; the binary64 format used throughout this book is unchanged from the 1985 version.

1.1. The IEEE 754 binary64 model

A binary64 floating-point number consists of a sign bit, an eleven-bit biased exponent, and a fifty-two-bit fraction (the *mantissa* or *significand*); when interpreted as a normalised number, the value is

$$x = (-1)^s \cdot 2^{e-1023} \cdot \left(1 + \frac{f}{2^{52}}\right),$$

where s is the sign, e is the unsigned exponent, and f is the fifty-two-bit fraction interpreted as an unsigned integer. The range of representable positive numbers spans roughly 2.2×10^{-308} to 1.8×10^{308} ; values outside that range overflow to infinity or underflow to a denormalised representation that gracefully degrades precision².

which the leading implicit "1" of the significand is replaced by a "0". They fill the gap between the smallest normal number and zero. Their handling is the slowest path through most FPUs; some real-time DSP code disables them via the FTZ ("flush to zero") flag.

The *machine epsilon* is the gap between 1 and the next representable number, which is $2^{-52} \approx 2.22 \times 10^{-16}$ for binary64. Every elementary arithmetic operation (+, −, *, /, sqrt) is required by the standard to produce the *correctly rounded* result – the unique nearest representable number to the true mathematical answer, with ties broken to even.

-1p

Programmer's rule of thumb: a single double-precision arithmetic operation introduces at most half a ULP of error; a hundred such operations in sequence can compound to a few hundred ULPs in the worst case, but is usually closer to a handful.

1.2. Absolute, relative, and propagated error

Given a true value x and a computed approximation $\hat{x}$, the *absolute error* is $|\hat{x} - x|$ and the *relative error* is $|\hat{x} - x| / |x|$ (defined for $x \neq 0$). Relative error is the more useful measure for floating-point work: it is dimensionless, roughly invariant under scaling, and bounded above by the machine epsilon for any single correctly-rounded operation.

When errors compound across a chain of operations, the worst-case relative error grows linearly with the number of operations only in well-conditioned algorithms; ill-conditioned algorithms can see exponential blow-up. The *condition number* of a problem – the ratio of relative-output-error to relative-input-error in the worst case – quantifies this sensitivity.**Theorem 1.1:** If $x \sup 1, x \sup 2$,

¹The current revision is IEEE 754-2019, which extended the

²Denormalised (or *subnormal*) numbers are those for

$\dots, x_n$ are computed in binary64 with each operation correctly rounded, the relative error of any single result is at most ϵ in the worst case, where ϵ is machine epsilon.

1.3. Convergence of iterative methods

Most numerical algorithms are *iterative*: starting from an initial guess x_0 , they produce a sequence $x_0, x_1, x_2, \dots$ that (we hope) tends to the true answer x^* . The standard convergence rates are:

- **Linear convergence:** $|x_{n+1} - x^*| \leq C |x_n - x^*|$ for some constant $0 < C < 1$. The error shrinks by a constant factor per step.
- **Quadratic convergence:** $|x_{n+1} - x^*| \leq C |x_n - x^*|^2$. The number of correct digits roughly doubles per step.
- **Cubic convergence:** the error term goes as the cube of the previous error. Rare in practice; Halley's method is the classic example.

Newton's method (chapter 2) is the workhorse of practical root-finding precisely because it achieves quadratic convergence on well-behaved problems while costing only one function and one derivative evaluation per step.

Chapter 2. Root-finding by Newton's method

Given a continuously differentiable function $f : \mathbb{R} \rightarrow \mathbb{R}$ and an initial guess x_0 close to a root x^* where $f(x^*) = 0$, *Newton's method* updates the guess by

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Geometrically each step replaces f with its tangent line at x_n and takes the root of that tangent as the next iterate. The method is the gold standard for smooth scalar root-finding: when it works it doubles the number of correct digits per step, and when it fails it usually fails fast and visibly.

2.1. Derivation from the Taylor expansion

Expanding f in a Taylor series about x_n and truncating after the linear term gives

$$f(x^*) \approx f(x_n) + f'(x_n)(x^* - x_n).$$

Setting the left side to zero (since x^* is a root) and solving for x^* produces the update formula. The error analysis follows by Taylor-expanding to second order: if f' is non-zero at the root and f'' is bounded near the root, the local error satisfies

$$x_{n+1} - x^* = -\frac{f''(\xi_n)}{2f'(x_n)}(x_n - x^*)^2,$$

which is the quadratic convergence claim made informally above.

2.2. Convergence theorem and pitfalls

The standard convergence theorem for Newton's method appears in every introductory text [3]:

Theorem 2.1: Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be twice continuously differentiable in a neighbourhood U of a root x^* where $f'(x^*) \neq 0$. Then there exists $\delta > 0$ such that for any initial guess x_0 with $x_0 - x^* < \delta$, Newton's iterates converge quadratically to x^* .

The pitfalls of Newton's method are well-documented:

1. *Divergence from a poor initial guess.* Far from the root the linearisation may not be a good approximation, and the iterates can wander indefinitely.
2. *Zero derivative at an iterate* ($f'(x_n) = 0$). The update formula divides by zero; the iteration stalls or diverges.
3. *Multiple roots.* If x^* is a root of multiplicity $m > 1$, $f'(x^*) = 0$ as well, and the convergence degrades to linear with rate $(m-1)/m$.
4. *Cycles.* For carefully chosen f and x_0 , the iterates can cycle with period two; the classic textbook example is $f(x) = x^3 - 2x + 2$ with $x_0 = 0$.

In all of these the safety net is a *fallback* to a slower-but-globally-convergent method (bisection, secant) when the Newton step is non-finite or moves the iterate outside a trusted bracket.

2.3. A worked example

To find the positive root of $f(x) = x^2 - 2$ (that is, $\sqrt{2}$), take $x_0 = 1.5$ and iterate:

n	x_n	f(x_n)	digits
0	1.5000000000000000	2.50e-01	1
1	1.4166666666666667	6.94e-03	3
2	1.4142156862745099	6.01e-06	6
3	1.4142135623746899	4.51e-12	11
4	1.4142135623730951	4.44e-16	16

The number of correct digits roughly doubles per step until the last step, where it hits the machine-precision floor. This is the textbook signature of quadratic convergence; in production code the iteration is terminated when $|f(x_n)|$ falls below a problem-specific tolerance or when the relative change between successive iterates is below a small multiple of machine epsilon.

2.4. Two engines, side by side

A useful exercise: implement Newton's method twice – once in straight-line numerical code, once in symbolic form – and run them on the same input to confirm the formula matches the analytic derivative. The numerical version (left) and the symbolic-derivative version (right) agree to all digits on smooth inputs, but the numerical version is faster and the symbolic version is more robust when the user supplies a typo in f' .

```
9p def newton(f, df, x0, tol=1e-12):
```

```
    x = x0
    for k in range(50):
        fx = f(x)
        if abs(fx) < tol:
            return x, k
        x = x - fx / df(x)
    raise NoConvergence()
```

The straight-line implementation above is the version that appears in every introductory numerical-methods course: a hard iteration count cap, a residual-based stopping criterion, and a divide-by-derivative step. In production code the $fx / df(x)$ step is replaced by a safeguarded variant that falls back to bisection when the Newton step takes the iterate outside a trusted bracket [4].

Chapter 3. Quadrature: integrating numerically

Definite integration is the second classical problem of numerical analysis after root-finding. Most integrands of practical interest – the Gaussian probability density, the Bessel functions, the integrals arising in finite-element analysis – have no closed-form antiderivative, and even those that do are often easier to compute by quadrature than by analytic evaluation. This chapter develops the two simplest *composite* rules (trapezoidal and Simpson's), their convergence behaviour, and the conditions under which they fail.

3.1. The composite trapezoidal rule

Given a continuous function f on a closed interval $[a, b]$ and a partition of that interval into n equal subintervals of width $h = (b-a)/n$ with nodes $x_i = a + ih$, the *composite trapezoidal rule* approximates the integral by summing the areas of the trapezoids fitted under the piecewise-linear interpolant of f :

$$T_n(f) = \frac{h}{2} \left[f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n) \right].$$

The error analysis is by Taylor expansion about each midpoint: for $f \in C^2[a, b]$,

$$\int_a^b f(x) dx - T_n(f) = -\frac{(b-a)h^2}{12} f''(\xi), \quad \xi \in (a, b).$$

The rule is therefore *second-order accurate*: halving h quarters the error. The cost is one function evaluation per node, with the endpoints reused if successive refinements share their grid.

3.2. Simpson's rule and higher orders

Simpson's 1/3 rule replaces the piecewise-linear interpolant with piecewise-quadratic on triples of nodes; the resulting composite rule is fourth-order accurate. Higher-order rules (Newton-Cotes of degree 5, 7, 9) exist but are not in routine use – their weight tables include large coefficients with mixed signs that amplify roundoff, and adaptive quadrature with Simpson's rule as the inner kernel usually wins on both accuracy and robustness.

3.3. Adaptive quadrature

When the integrand has localised features (sharp peaks, near-singular endpoints, oscillation over a narrow region), uniform spacing wastes function evaluations on the smooth parts. *Adaptive* quadrature subdivides only where the error estimate exceeds a tolerance, recursively narrowing in on the difficult regions. The QUADPACK library [5] is the de facto reference implementation; modern environments (SciPy's `scipy.integrate.quad`, MATLAB's `integral`, R's `integrate`) wrap QUADPACK directly.

A useful consequence of Richardson extrapolation, due originally to Romberg, is that successive trapezoidal estimates at n and $2n$ subintervals can be combined to produce a Simpson-rule result at no extra function evaluations. The reverse direction also holds: $S_n - T_n$ is a cheap and surprisingly tight estimate of the trapezoidal error.

3.4. Endpoint singularities

The error theorems above assume f is smooth on the closed interval. When f has a singularity at an endpoint – a logarithmic divergence, a square-root cusp, a pole – both Newton-Cotes rules degrade. For $f(x) = \sqrt{x}$ on $[0, 1]$, both trapezoidal and Simpson exhibit only $O(n^{-3/2})$ convergence; for $f(x) = \log x$, the trapezoidal rule is limited to $O(n^{-2})$ and Simpson does only marginally better.

The remedies are well known. A *graded mesh* (concentrating nodes near the singularity), a *change of variables* ($u = \sqrt{x}$ turns a square-root singularity into a smooth integrand), or a *specialised rule* designed for the singularity class [6] all recover the smooth-case rates.

Chapter 4. References

1. N. J. Higham. *Accuracy and Stability of Numerical
2. IEEE Computer Society. *IEEE Standard for
3. J. Stoer and R. Bulirsch. *Introduction to Numerical
4. R. P. Brent. *Algorithms for Minimization without
5. R. Piessens, E. de Doncker-Kapenga, C. W.
6. J. N. Lyness and B. W. Ninham. Numerical quadrature

Algorithms*. Second edition. SIAM, Philadelphia, 2002.

Floating-Point Arithmetic*. IEEE Std 754-2019, IEEE, New York, 2019.

Analysis*. Third edition. Springer-Verlag, New York, 2002.

Mathematics of Scientific Computing*. Third edition. Brooks/Cole, Pacific Grove, 2002.

Derivatives*. Prentice-Hall, Englewood Cliffs, 1973.

Uberhuber, and D. K. Kahaner. *QUADPACK: A Subroutine Package for Automatic Integration*.

Springer-Verlag, Berlin, 1983.

and asymptotic expansions. *Mathematics of Computation*, 21(98):162-178, 1967.

Chapter 5. Index

-1p **Absolute error** 1 ; ULP 1 ; epsilon (machine) 1
Adaptive quadrature 3 ; QUADPACK 3
Binary64 1 ; significand 1 ; exponent 1
Condition number 1
Convergence linear 1 ; quadratic 1 ; cubic 1
Denormalised numbers 1 ; FTZ flag 1
IEEE 754 1
Machine epsilon 1
Newton's method 2 ; convergence theorem 2 ;
 pitfalls 2 ; worked example 2 ; safeguarded 2
Quadrature adaptive 3 ; composite 3 ; Simpson 3 ;
 trapezoidal 3 ; endpoint singularity 3
Relative error 1
Richardson extrapolation 3 ; Romberg 3
Roots, multiple 2
Trapezoidal rule 3 ; error theorem 3